NETWORK DESIGN · GCP · EGRESS

Agent Gateway Egress Design

The enterprise egress design for Gemini Enterprise and Agent Runtime (Agent Engine) agents. Every agent call leaves through Google's Agent Gateway, hosted in the hub that already runs Apigee X. Apigee then reaches internal APIs, and internet APIs through Zscaler. No new projects, no new VPCs and no new VMs. The design keeps your rules: Zscaler for internet egress, no VPC peering, NAT only on Zscaler subnets, Infoblox for all DNS, every CIDR in IPAM, prod and nonprod apart, east and west in every layer. Ingress is covered separately.

Draft v5 for review · 22 Oct 2026 · scope: GCP egress for Gemini Enterprise and Agent Runtime

Quick answers

- **One PSC connection per gateway?** Yes. Each Agent Gateway has exactly one network attachment (its PSC interface into a VPC). Google also allows several gateways to share one attachment or subnet.
- **Can many agent platforms use one gateway?** Yes for Agent Runtime: agents from any project in the same region can bind to one egress gateway. For Gemini Enterprise, each app is bound to one gateway in the region its location dictates (global or us → `us-central1`, eu → `europe-west1`). Google's example puts that gateway in the app's own project.
- **Is PSC still needed to reach Apigee?** Not for gateways hosted in Apigee's hub. Their PSC interface lands in the same VPC as Apigee's load balancer, so they reach it directly. Only a gateway in a different hub would need a PSC endpoint to Apigee's service attachment.
- **Can the gateways live in the hub host project?** Yes. Google documents both an attachment in the host project and one in a service project. The host-project option gives the gateway's service agent network-admin rights on the host project, which can be cut down to a custom role.

Terms used

TERM	MEANING HERE
Egress gateway	A Google-managed Agent Gateway in `AGENT_TO_ANYWHERE` mode. Every outbound call from the agents bound to it passes through it. One per environment and region, named like `agw-prod-east`.

TERM	MEANING HERE
Landing subnet	The subnet in a hub VPC where a gateway's PSC interface lands. Traffic leaving the gateway appears here, with source addresses from this range.
Apigee hub	The hub (called Hub A here) whose host project holds Apigee X's MIG bridge and load balancer. The egress gateways live in this hub's host project.
Agent API hostname	The Apigee hostname agents call, for example `agents-api.prod.corp.internal`. Each external or internal API sits behind it as an Apigee API proxy.
Binding	A setting on an agent or Gemini Enterprise app that names its egress gateway, plus an IAM grant. It isn't a network connection.

The design in one page

Gateways in the Apigee hub

Four egress gateways for Agent Runtime (prod and nonprod, east and west) live in Hub A's host project. Their PSC interfaces land on small landing subnets in Hub A's prod and nonprod VPCs. Gemini Enterprise gets its own gateway in the region its app location requires.

Every platform binds; nothing is wired

Agents in any business unit's project, and Gemini Enterprise apps, bind to the gateway for their environment and region. Agent projects stay attached to their own hub, unchanged.

Apigee is the exit

Gateways call the agent API hostname, which Infoblox resolves to Apigee's load balancer in the same VPC. Apigee runs one API proxy per approved destination, holds the credentials, and calls internal APIs directly and internet APIs through Zscaler.

Nothing else is reachable

The landing subnets carry no Zscaler tag, so there's no internet route. A firewall allows them to reach Apigee and Google APIs only. A name that isn't an approved API gets nowhere.

Design rules

1. All agent and Gemini Enterprise egress goes through an Agent Gateway. No agent keeps a direct PSC interface.
2. Agents reach the internet only through Apigee API proxies. Arbitrary websites and downloads are not reachable.

3 Apigee's calls to the internet go through Zscaler. This must be confirmed before go-live.

4 Prod and nonprod never share a gateway, a landing subnet or an Apigee environment.

5 East and west are built identically, and each region works on its own.

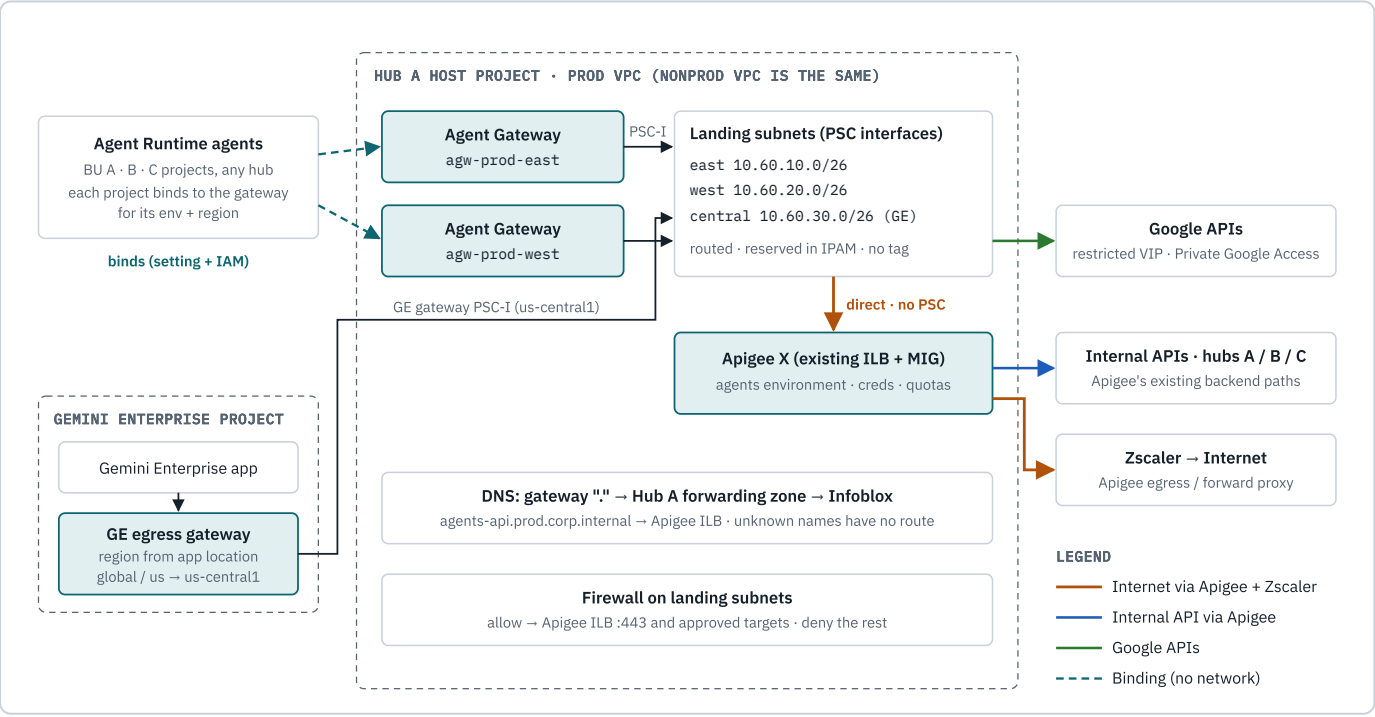
6 Landing subnets never carry the Zscaler tag. A firewall limits them to Apigee and Google APIs.

7 All DNS goes through Infoblox. Nothing resolves publicly from GCP.

8 Every range is reserved in IPAM. Landing subnets are routed hub ranges, so they must be enterprise-unique.

Where the gateways live

All Agent Runtime egress gateways sit in the Apigee hub's host project, one per environment and region. Each lands in that environment's VPC, next to Apigee. Business-unit agents in any hub bind across projects, which Google supports for egress as long as the agent and gateway are in the same region. Gemini Enterprise's gateway follows the app's location, which may not be east or west.



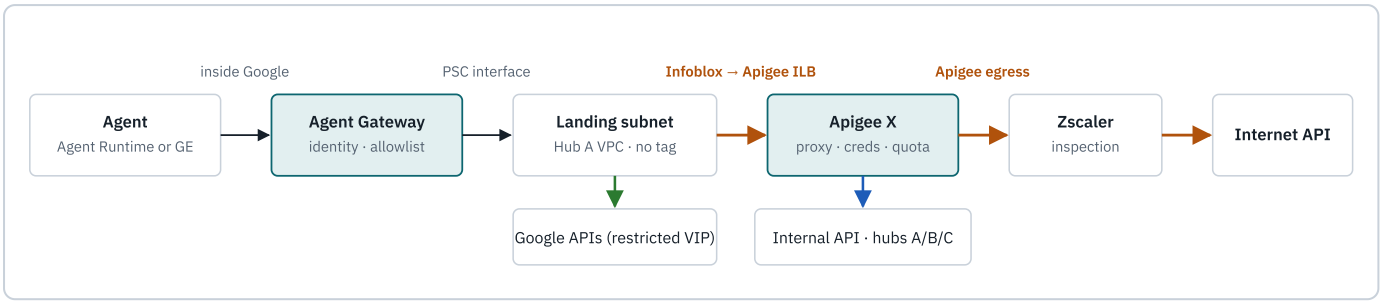
Hub A's nonprod VPC holds the matching nonprod gateways, landing subnets and nonprod Apigee. No new projects or VPCs are created; the only additions to Hub A are the gateways, landing subnets, a DNS forwarding zone if missing, and firewall rules.

GATEWAY	PROJECT	LANDS IN	WHO BINDS TO IT
`agw-prod-east`	Hub A host	Hub A prod VPC, east landing subnet	Prod Agent Runtime agents in east, all BUs
`agw-prod-west`	Hub A host	Hub A prod VPC, west landing subnet	Prod Agent Runtime agents in west, all BUs
`agw-nonprod-east`	Hub A host	Hub A nonprod VPC, east landing subnet	Nonprod Agent Runtime agents in east, all BUs
`agw-nonprod-west`	Hub A host	Hub A nonprod VPC, west landing subnet	Nonprod Agent Runtime agents in west, all BUs
`agw-ge-prod`	Prod Gemini Enterprise project	Hub A prod VPC, landing subnet in the GE region	The prod Gemini Enterprise app
`agw-ge-nonprod`	Nonprod Gemini Enterprise project	Hub A nonprod VPC, landing subnet in the GE region	The nonprod Gemini Enterprise app

Compared with a separate gateway project

	GATEWAYS IN HUB A HOST PROJECT (V5, CHOSEN)	SEPARATE GATEWAY PROJECTS WITH THEIR OWN VPCS (V4)
New projects	None	Two (prod, nonprod)
New VPCs	None	Four small isolated VPCs
Reaching Apigee	Directly, same VPC	PSC endpoint per region
DNS	Infoblox, through the hub's existing resolver path	Separate private Cloud DNS view
Isolation	Landing subnets sit in a routed hub, so a firewall must limit them	Isolated by design; nothing routable
Access to the host project	Agent platform team and the gateway's service agent need rights in the host project	Host project untouched
IPAM	Landing subnets are routed and enterprise-unique	Isolated blocks

How a call flows



The gateway only ever sends packets to Apigee and Google APIs. Apigee makes every other call.

Agent → internet SaaS API

LEG 1 · INSIDE GOOGLE

1. The agent calls `https://agents-api.prod.corp.internal/salesforce/…` with its Apigee credential. Its binding sends the call to `agw-prod-east`.
2. The gateway checks the agent's identity and its `iap.egressor` grant on the agent API hostname.
3. The gateway resolves the name through its root DNS peering. Hub A's root forwarding zone asks Infoblox, which answers with Apigee's load balancer IP.

LEG 2 · HUB A PROD VPC

4. The gateway connects from its landing-subnet address to Apigee's load balancer on 443. It's a subnet route in the same VPC, so no PSC endpoint is needed. The firewall allows it.
5. The gateway checks Apigee's certificate against the corporate CA in its `trustConfig`. The MIG bridge passes the call to the Apigee runtime.

LEG 3 · APIGEE → ZSCALER → INTERNET

6. The `/salesforce` proxy checks the agent's credential, applies its quota and swaps in the Salesforce credential.
7. Apigee calls `api.salesforce.com` through its egress path, which goes through Zscaler. Zscaler inspects it, and the response comes back the same way.

Agent → internal API in another hub

1. The agent calls `https://agents-api.prod.corp.internal/orders/…`.
2. Legs 1 and 2 are the same as above.
3. The `/orders` proxy calls the backend in Hub B over Apigee's existing backend path, which already serves other consumers.
4. No Zscaler, no new PSC work.

Agent or GE → Gemini and Google APIs

1. The model call (for example `{region}-aiplatform.googleapis.com`) goes through the gateway too, so each Google API hostname must be allowlisted (`iap.resources.egressViaIAP`).
2. Infoblox answers `googleapis.com` with the restricted VIP, as it already does for hub workloads.
3. Hub A's route for `199.36.153.4/30` and Private Google Access on the landing subnet carry it to Google, without Apigee or Zscaler.

Anything not approved

1. A real internet hostname resolves to a public IP. The landing subnet has no Zscaler tag, so there's no route: dropped.
2. An internal hub address: the firewall on the landing subnets denies it.
3. An agent API hostname without a grant: denied by the gateway. An unsubscribed API: rejected by Apigee.

DNS through Infoblox

Agent Gateway uses public recursive DNS unless its connectivity template peers domains into the VPC it lands in. Google documents a root peering for this: the template peers `.`, and a **private forwarding zone for the root** in the host project that owns the VPC sends queries to an internal enterprise resolver. Here that resolver is your Infoblox grid members in Hub A, so the gateway resolves exactly like every other Hub A workload.

- **Zone location:** Google requires the managed zone to be in the host project that owns the target network, which is Hub A's host project. Cross-project DNS peering isn't supported in this setting.
- **If Hub A already sends everything to Infoblox** through a root forwarding zone, the gateway reuses it. If Hub A uses a different method, such as a DNS server policy, adding a root forwarding zone changes how Hub A's own VMs resolve too. Agree it with the DNS team first.
- **New records in Infoblox:** `agents-api.prod.corp.internal` and `agents-api.nonprod.corp.internal` → the matching Apigee load balancer. `googleapis.com` → restricted VIP is probably already there.
- **Unknown names are harmless.** If Infoblox resolves a public name, the gateway still has no route to it.
- **Don't use an authoritative private root zone here.** In a hub, it would break resolution for every Hub A workload.

Gemini Enterprise

Gemini Enterprise uses Agent Gateway in egress mode only. Binding an app sends **all** its traffic through the gateway, including LLM calls.

ITEM	WHAT GOOGLE DOCUMENTS	DESIGN DECISION
Gateway region	Follows the app's location: global or us → `us-central1`; eu → `europe-west1`	Add a landing subnet in that region to Hub A's prod and nonprod VPCs. If it isn't east or west, it's a third region for this one purpose
Gateway project	Google's example has the gateway and its registry in the app's project	One gateway per GE app project (prod, nonprod). Its attachment uses a Hub A shared subnet: the service-project pattern Google recommends for Shared VPC
Binding	App field `agentGatewaySetting.defaultEgressAgentGateway.name`, or Security › Configuration › Agent Gateway configuration in the console	Set by the GE platform team after the gateway passes dry run
Registries	Up to two per gateway (one global, one regional or multi-regional). `us` and `eu` multi-region registries don't allow manual registration	Register MCP servers, A2A agents and endpoints in the global registry
MCP servers and A2A agents	Must be imported from Agent Registry into the app	Internal MCP servers sit behind Apigee where possible, or get a PSC endpoint in Hub A if not
Data connectors	Opt in per connector (`use_agent_gateway_egress`)	**Decision needed:** connectors that call SaaS hostnames directly can't be redirected to Apigee URLs. They either stay off the gateway (Google-managed egress) or need another internet path
Org policy	A custom constraint can block apps from binding to gateways	Use it to stop unapproved bindings, allowing only the platform gateways

`verify in lab` Two things to prove: a gateway in `us-central1` reaching Apigee's east or west load balancer in the same VPC (Apigee's load balancer needs global access), and whether a GE app can bind to a gateway in another project.

Network and security controls

- **Firewall on landing subnets** (network firewall policy on each Hub A VPC):
  - Allow landing ranges → Apigee load balancer on TCP 443.

    - Allow → any approved direct PSC endpoints.
    - Deny landing ranges → everything else, including RFC 1918 and on-prem ranges.

  `verify in lab` Check that egress rules matched on source range apply to PSC-interface traffic. If they don't, also restrict on the Apigee MIG's ingress rules.

- **No tag, ever.** The landing subnets never get the Zscaler tag. They can't reach the internet even if a policy is wrong, because the hub's only internet route requires the tag.
- **Host-project access.** Two kinds of access are needed in Hub A's host project:
    - The gateway's service agent needs to update its network attachment. Use Google's custom-role option (`compute.networkAttachments.get` , `compute.networkAttachments.update` , `compute.regionOperations.get`) plus `roles/dns.peer` , not full Network Admin.
    - The agent platform team needs the gateway, registry, Certificate Manager and IAP policy roles, scoped by IAM conditions where possible. Alternatively, the network team owns the gateway resources and the platform team requests changes through code review.
- **Registry and grants live in the host project.** Every agent's registration and `iap.egressor` grant is managed there. Keep them in infrastructure-as-code.
- **VPC Service Controls:** if agent projects are in a perimeter, the Hub A host project and the gateway projects must be in the same perimeter, according to Google's Shared VPC guidance.
- **No Cloud NGFW Enterprise** on Hub A's VPCs. Agent Gateway returns 503 on those networks.

Resource and IP plan

Landing subnets live in routed hub VPCs, so they come from the enterprise IPAM like any hub subnet, and must not overlap anything reachable over Interconnect. Each gateway needs at least a /28 (12 usable addresses). A /26 leaves room for more gateways to share the same attachment later.

LANDING SUBNET	VPC	REGION	EXAMPLE RANGE	USED BY
`snet-agw-landing-prod-east`	Hub A prod	east	10.60.10.0/26	`agw-prod-east`
`snet-agw-landing-prod-west`	Hub A prod	west	10.60.20.0/26	`agw-prod-west`
`snet-agw-landing-prod-ge`	Hub A prod	GE region	10.60.30.0/26	`agw-ge-prod`
`snet-agw-landing-nonprod-east`	Hub A nonprod	east	10.61.10.0/26	`agw-nonprod-east`

LANDING SUBNET	VPC	REGION	EXAMPLE RANGE	USED BY
`snet-agw-landing-nonprod-west`	Hub A nonprod	west	10.61.20.0/26	`agw-nonprod-west`
`snet-agw-landing-nonprod-ge`	Hub A nonprod	GE region	10.61.30.0/26	`agw-ge-nonprod`

WHERE	RESOURCE	KEY SETTINGS
Hub A host	Landing subnets	Private Google Access on; never tagged; firewall-restricted
Hub A host	Network attachments	One per landing subnet, `ACCEPT_AUTOMATIC`
Hub A host	Root forwarding zone (if missing)	`.` → Infoblox grid members (private routing), authorized for each Hub A VPC
Hub A host	Agent Gateways ×4 + connectivity templates	`AGENT_TO_ANYWHERE`, `CENTRALIZED`, `vpcEgress: ALL_TRAFFIC`, DNS peering `.`, corporate CA in `trustConfig`
Hub A host	Agent Registry + IAP authz	Global + regional registry; entries = agent API hostnames + Google API hostnames; start in `DRY_RUN`
GE projects	GE gateway + template + registry	As above, in the GE region; attachment on the GE landing subnet
Apigee (existing)	Agents environment and environment group	Hostname `agents-api.{env}.corp.internal`; streaming on; longer timeouts; forward proxy to Zscaler if needed
Infoblox	Records	Agent API hostnames → Apigee load balancer; restricted VIP for `googleapis.com`

Implementation steps

Build nonprod first (east, west, then the GE gateway), then prod. Gateways are identical apart from names, so write one Terraform module and instantiate it per gateway. The commands show `agw-prod-east`. `documented` marks steps from Google's docs. `verify in lab` marks steps to prove in nonprod first.

PHASE 0 Decisions and approvals

Network · Apigee · Security · Zscaler · DNS · GE teams

- **Network team:** approve gateways and landing subnets in Hub A's host project, and the host-project access described above.

- **Apigee team:** host the agents environment in prod and nonprod, in both regions. Confirm capacity, an SLO, and global access on the load balancer if the GE region differs.
- **Zscaler team:** confirm Apigee's internet calls go through Zscaler. If not, provide an explicit-proxy endpoint for the agents environment's forward proxy.
- **DNS team:** confirm how Hub A resolves today, and approve the root forwarding zone to Infoblox if it's missing.
- **GE team:** confirm the app's location (global, us or eu) and decide which data connectors use the gateway.
- **IPAM:** reserve the six landing subnets.
- **Org policy:** `constraints/iam.managed.disableAccessPolicyBindings` off where gateways live. documented

PHASE 1 Hub A preparation Network team

```
# Landing subnet (repeat per gateway)
gcloud compute networks subnets create snet-agw-landing-prod-east --project=HUB_A_HOST --
network=HUB_A_PROD_VPC --region=EAST --range=10.60.10.0/26 --enable-private-ip-google-access

# Root forwarding zone to Infoblox (only if Hub A doesn't already have one)
gcloud dns managed-zones list --project=HUB_A_HOST
gcloud dns managed-zones create pz-root-to-infoblox --project=HUB_A_HOST --dns-name=. --
visibility=private --networks=HUB_A_PROD_VPC --private-forwarding-
targets=INFOBLOX_GRID_IP_1,INFOBLOX_GRID_IP_2 --description="all names to Infoblox"

# Network attachment for the gateway
gcloud compute network-attachments create na-agw-prod-east --project=HUB_A_HOST --region=EAST --
subnets=snet-agw-landing-prod-east --connection-preference=ACCEPT_AUTOMATIC
```

- Firewall policy rules for the landing subnets, as in controls.
- Check that Hub A's route for `199.36.153.4/30` to the default internet gateway exists.
- Grant the gateway service agent (`service-HUB_A_NUMBER@gcp-sa-agentgateway.iam.gserviceaccount.com`) the custom attachment role and `roles/dns.peer` . documented

---

PHASE 2 Apigee agents environment Apigee team

1. Create an `agents` environment per Apigee org, attached to the east and west instances.
2. Create an environment group with hostname `agents-api.prod.corp.internal` and a certificate from the corporate CA. Infoblox points the name at Apigee's load balancer.
3. If Apigee's internet calls don't already go through Zscaler, set the forward proxy on this environment only. verify in lab
4. Allow the landing ranges in the MIG bridge's ingress firewall.
5. Build a template API proxy (agent credential check → quota → credential injection → target, streaming on, longer timeouts). Create one proxy per destination from it, plus API products and one developer app per

agent or BU.

PHASE 3 Agent Runtime gateways (×4) Agent platform team

```
gcloud certificate-manager trust-configs import tc-corp-ca-east --project=HUB_A_HOST --location=EAST -
-source=tc-corp-ca.yaml

# connectivity-template.yaml
accessPath: AGENT_TO_ANYWHERE
deploymentModel: CENTRALIZED
egressNetworkConfig:
  networkAttachment: projects/HUB_A_HOST/regions/EAST/networkAttachments/na-agw-prod-east
  vpcEgress: ALL_TRAFFIC
  dnsPeeringConfig:
    domains: ["."]                # resolved by Hub A's root forwarding zone → Infoblox
    targetNetwork: projects/HUB_A_HOST/global/networks/HUB_A_PROD_VPC
tlsConfig:
  trustConfig: projects/HUB_A_HOST/locations/EAST/trustConfigs/tc-corp-ca-east
  additionalRoots: PUBLICLY_TRUSTED_ROOTS

# agent-gateway.yaml → gcloud network-services agent-gateways import agw-prod-east --
project=HUB_A_HOST --location=EAST --source=agent-gateway.yaml
name: agw-prod-east
googleManaged:
  governedAccessPath: AGENT_TO_ANYWHERE
agentConnectivityTemplate: projects/HUB_A_HOST/locations/EAST/agentConnectivityTemplates/act-agw-prod-
east
registries:
  - //agentregistry.googleapis.com/projects/HUB_A_HOST/locations/global
  - //agentregistry.googleapis.com/projects/HUB_A_HOST/locations/EAST

# authz extension: iap.googleapis.com, failOpen false, timeout 1s, iapPolicyVersion "V2",
iamEnforcementMode "DRY_RUN"
# authz policy:   policyProfile REQUEST_AUTHZ, action CUSTOM, target = the gateway
```

- verify in lab The field names come from Google's docs; check the nesting and the import commands against the current API reference.
- Register the agent API hostname and the Google API hostnames agents use: exact names, with no wildcards.

> **Gate:** deploy a test agent bound to `agw-nonprod-east` that calls a test Apigee proxy (target: a public test site) and Gemini. Pass: both calls work in dry run. Apigee, Zscaler and the gateway logs show the calls, and a direct call to a public hostname fails.

PHASE 4 Gemini Enterprise gateways (×2) GE team · Agent platform team

1. Attach each GE project to Hub A as a service project. Share the GE landing subnet with it only.
2. In the GE project, create the network attachment on the shared GE landing subnet, then the TrustConfig, template and gateway in the GE region. The template peers `.` to the Hub A VPC. This is Google's recommended Shared VPC pattern. documented

3. Register MCP servers, A2A agents and endpoints in the global registry, and import them into the app.
4. Bind the app after dry run. `verify in lab` (check the API version and path)

```
curl -X PATCH -H "Authorization: Bearer $(gcloud auth print-access-token)" -H "Content-Type:
application/json"
"https://discoveryengine.googleapis.com/v1alpha/projects/GE_PROJECT/locations/global/collections/default_
updateMask=agentGatewaySetting.defaultEgressAgentGateway.name" -d '{"agentGatewaySetting":
{"defaultEgressAgentGateway":{"name":"projects/GE_PROJECT/locations/us-central1/agentGateways/agw-ge-
prod"}}}'
```

PHASE 5 Bind Agent Runtime agents

BU agent teams · Agent platform team

1. Grant each agent project's Runtime service agent a custom role on Hub A's host project with `networkservices.agentGateways.get` and `networkservices.operations.get`. `documented`
2. Change the agent's tools to call the agent API hostname and send its Apigee credential.
3. Deploy or update the agent with the binding and `AGENT_IDENTITY`, and remove `psc_interface_config`.
4. Register the agent and grant `roles/iap.egressor` on the agent API hostname and Google API hostnames.

```
remote_agent = client.agent_engines.create(
    agent=app,
    config={
        "identity_type": "AGENT_IDENTITY",
        "agent_gateway_config": {
            "agent_to_anywhere_config": {
                "agent_gateway": "projects/HUB_A_HOST/locations/EAST/agentGateways/agw-prod-east"
            }
        },
        "env_vars": {"AGENTS_API_BASE": "https://agents-api.prod.corp.internal"},
    },
)
```

PHASE 6 Test, then enforce

Agent platform · Apigee · Security

- Run each gateway in `DRY_RUN` for at least one business cycle. Review IAP decisions, Apigee analytics and firewall logs.
- Run the test plan, then switch to enforce. Then move on to the next gateway.

Rollout and migration

1. **Pilot:** one nonprod BU, `agw-nonprod-east`, two or three APIs in Apigee.
2. **Nonprod complete:** the remaining nonprod gateways, all BUs, and the nonprod GE app.
3. **Prod:** east, then west, then the GE app, each in dry run first.

- **Agents created before 29 April 2026** can't bind to a gateway. Redeploy them.
- **Changing an agent's binding archives all its revisions**, and that can't be undone. Bind once, in a planned change.
- **Change tools before binding.** After binding, an agent can only reach Apigee and Google APIs.
- **Binding a GE app immediately reroutes all its traffic.** Allowlist its Google API hostnames first, or the app stops working.
- **Tell the SOC.** Security Command Center's Agent Engine Threat Detection is turned off for bound agents.

Test plan

TEST	PASS
Agent → SaaS API through Apigee	Works; Apigee and Zscaler logs show it; gateway log shows the agent identity
Agent → public hostname directly	Fails: no route from the landing subnet
Agent → internal Hub A address directly	Blocked by the landing-subnet firewall
Agent without an `iap.egressor` grant	Denied by the gateway
Agent → API it isn't subscribed to	Rejected by Apigee (401/403)
Apigee egress source	Targets see Zscaler egress IPs, never Apigee's own NAT IPs
DNS	Gateway lookups reach Infoblox; nothing goes to a public resolver from GCP
GE app → LLM, MCP server, A2A agent	All work through `agw-ge-*` after binding
Streaming (MCP, LLM)	Streams through Apigee without buffering or timeouts
Nonprod → prod	Fails: different VPC, Apigee org and gateway
Region independence	Disabling east affects only east agents

Operating model

AREA	OWNER	RUNS
Landing subnets, firewall, DNS zone, attachments	Network team	Hub A changes, IPAM, Infoblox records
Egress gateways, registry, grants	Agent platform team (in Hub A, through IaC)	Gateway module, onboarding pipeline, dry-run reviews
GE gateways and app binding	GE platform team	GE gateways, MCP and A2A imports, connector decisions
Apigee agents environment	Apigee team	API proxies, products, apps, credentials, quotas, capacity
Apigee internet egress	Apigee + Zscaler teams	Path to Zscaler and Zscaler policy

- **New agent:** binding, gateway role, registry entry, `iap.egressor` grant, Apigee app and credential.
- **New API for agents:** Apigee proxy from the template, plus a product subscription. No network or DNS change.
- **New business unit:** nothing in the network. Its agent projects bind to the existing gateways.
- **Scale limits:** 5,000 registry resources per gateway; two registries per gateway; one egress gateway per agent project and region.
- **Monitor:** gateway 498 and 503 rates; IAP denials; landing-subnet firewall denies; Apigee latency, errors and quota hits; Zscaler traffic from Apigee.
- **CA rotation:** update the corporate CA in every TrustConfig before it expires. It isn't automated.

Open questions

1. **Does Apigee's internet egress go through Zscaler today?** If it uses Apigee's own NAT, the agents environment needs a forward proxy to a Zscaler explicit endpoint.
2. **Are Apigee prod and nonprod both in Hub A, in east and west?** Gateways in Hub A reach Apigee directly only if it's in the same VPC.
3. **How does Hub A resolve DNS today?** A root forwarding zone to Infoblox, or something else? This decides whether the gateway's root peering can use the existing setup.
4. **Where is the Gemini Enterprise app (global, us or eu)?** That sets the GE gateway region, and possibly a third region in Hub A.

5. **Which GE data connectors must use the gateway?** SaaS connectors can't be redirected to Apigee.
6. **Are all agent destinations APIs?** Anything else isn't reachable in this design.
7. **Who may change gateway resources in Hub A's host project?** The platform team through IaC, or the network team on request.

Sources

- Agent Gateway overview
- Set up Agent Gateway
- Set up VPC connectivity for Agent Gateway (Shared VPC roles, shared attachments, root DNS peering)
- Route Gemini Enterprise traffic through Agent Gateway
- Route Agent Runtime traffic through Agent Gateway
- Apigee: provisioning NAT IPs (default internet egress)